后训练学习计划 • 第 5 周（Phase 1 收尾 • 10/5-10/11 • 硬里程碑 M1：在 4090 上从零训一个 GPT）

本周定位：**Phase 1 的终点线，也是整条路线第一个拿得出手的作品**。你要在自己的账号、自己租的卡上，从一个空文件训出一个会写莎士比亚腔、会讲小学生故事的小 GPT，并演练一次完整的「训练发散 → 定位 → 修复」。交付物是三件：`m1-nanogpt` 仓库、博客《我从零训了一个 GPT》、Phase 1 闭卷自测通过。**这三件的交付时间不许顺延**（要顺延只能砍任务，见总览下的砍单顺序）。 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一 10/5	上周 gaps 收口 + nanoGPT 落地 + 参数量/显存手算	本地 CPU 冒烟跑通 + 手算表	6h
Day 2 周二 10/6	上卡：字符级莎士比亚正式训练 + 基线对比 + 断点续训	val loss <1.9 且采样可读 + baseline PPL 对比	6.5h
Day 3 周三 10/7	**训练不稳定排查演练（M1 自测标准）**	diagnosis.md：LR→精度→数据→初始化 四段定位	6.5h
Day 4 周四 10/8	BPE + TinyStories 数据管线 + 长跑启动	prepare 脚本 + 300 iters 冒烟 + 长跑挂起	6h
Day 5 周五 10/9	续训收口 + 采样参数对照 + repo 整理	采样对照表 ≥6 格 + `m1-nanogpt` 可复现	6.5h
Day 6 周六 10/10	闭卷自测 + 博客《我从零训了一个 GPT》成文发布	博客链接 + 闭卷 4 题	6h
Day 7 周日 10/11	补漏 + 闭卷复测 + **Phase 1 总验收** + 周 6 预习	验收 ≥6/7 项通过	4.5h

注：**本周卡时上限 12 小时，预估花费 ¥25-30**（4090 按量 ¥1.5-2.5/时，按 ¥2-2.5/时算）。分配：Day 2 约 2h（正式训 + 断点演练）、Day 3 约 3h（三次发散 run 都是短跑）、Day 4 约 2.5h（BPE 冒烟 + 长跑首段）、Day 5 约 2.5h（续训尾段 + 采样）、机动 2h。**超了怎么办**：Day 4 的 TinyStories 长跑把 `max_iters` 从 6000 砍到 3000，或整体降级为「只交付字符级莎士比亚模型 + TinyStories 冒烟曲线」，这两条都在计划里写死了降级出口，砍完必须能压回 12h。

注：本周最大风险是**你盯着进度条空等**。字符级模型 3000 iters 在 4090 上只要十几分钟，而 BPE 版要两小时以上——所有等待时间都要拿去写 NOTES 或做当天的下一个任务，别刷页面等卡。第二风险是**Day 3 的发散演练被跳过**（因为它不产出漂亮曲线）；它恰恰是面试最值钱的部分，「你遇到过 loss 变 NaN 吗，怎么查的」这题，本周之后你必须能答 5 分钟。

注：砍单顺序（实在做不完时按这个砍）：Day 5 任务 2 采样对照表（6 格 → 3 格）→ Day 4 任务 4 差异清单（延到 Day 6 博客里补一段）→ Day 2 任务 4 断点续训（延到 Day 4 一起做）→ Day 1 任务 4 手算表（只算不实测）。**不许砍**：Day 3 全部、Day 4 任务 1-3、Day 6 博客、Day 7 的 Phase 1 总验收。M1 的三件交付物不可延期。

二、开始前：环境、仓库与算账（约 40 分钟，10/4 前一晚做完）

1. 本地环境沿用第 1 周那个独立 venv（**不要装进 conda base**）。补装本周依赖：

```
source E:/study/posttrain/.venv/Scripts/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install numpy matplotlib tqdm wandb tiktoken
```

2. 建本周目录，并把 nanoGPT 放在 `E:/study/posttrain/` 下（周 3 已经 clone 过就直接 `cd` 进去用，别重复下载；它本身不进你的实验仓库）：

```
mkdir -p E:/study/posttrain/week05/{day1,day2,day3,day4,day5,day6}
mkdir -p E:/study/llm-posttraining-lab/m1-nanogpt
cd E:/study/posttrain
git clone https://github.com/karpathy/nanoGPT.git     # 已存在就 git -C nanoGPT pull
cd nanoGPT && pip install -r requirements.txt
```

`requirements.txt` 装失败就手动补：`pip install numpy torch tqdm wandb datasets tiktoken`。

3. AutoDL 账号里余额 ≥¥50（本周上限 ¥30，加上可能的重试）。**开卡前**把这张表抄进 `week05/day1/cost.md`，跑完回填实际值：

日期	任务	计划卡时	计划金额	实际卡时	实际金额
10/6	字符级正式训 + 断点演练	2.0h	¥4-5		
10/7	发散排查 3 段	3.0h	¥6-8		
10/8	BPE 冒烟 + 长跑首段	2.5h	¥5-6		
10/9	续训尾段 + 采样	2.5h	¥5-6		
机动	重试/重跑	2.0h	¥4-5		

4. 训练启动脚本模板（先写好再开卡，这是记账纪律）。存 `m1-nanogpt/run_char.sh`：

```
#!/bin/bash
# run_char.sh —— 字符级莎士比亚。GPU 占用 = 这一条命令的生命周期
# 用法：tmux new -s m1 -d './run_char.sh | tee /root/autodl-tmp/train_char.log'
source /etc/network_turbo            # AutoDL 学术加速，对 github/pypi 有效；失败忽略
export HF_ENDPOINT=https://hf-mirror.com
python train.py \
  --dataset=shakespeare_char \
  --scale_lr --batch_size=24 --block_size=256 \
  --n_layer=6 --n_head=6 --n_embd=384 \
  --learning_rate=6e-4 --max_iters=6000 --lr_decay_ratio=0.1 \
  --eval_interval=500 --eval_iters=200 --log_interval=50 \
  --decay=True --warmup_iters=100 --beta2=0.95 --grad_clip=1.0 \
  --dtype=bfloat16 --compile=False --out_dir=out-char6 \
  --wandb_log=True --wandb_project=m1-nanogpt
```

5. 把 `week04` 的 `gaps.md`、`NOTES.md` 复制一份到 `week05/day1/`，Day 1 任务 1 要用。

> 注：`/root/autodl-tmp` 是 AutoDL 的持久盘，`/root` 下的东西可能随镜像重置没了。**日志写持久盘、代码靠 `git push` 双备份**。checkpoint 也在持久盘（`--out_dir` 给绝对路径更稳）。关机不是销毁——任何时候不要点「销毁」，那会连数据盘一起清空。

Day 1 • 周一：上周 gaps 收口 + nanoGPT 落地 + 参数量手算

今日总览（6h）：今天不上卡。三件事必须完成：把 Week 4 欠的洞清掉几条（下面任务 1 逐条列了）、把 nanoGPT 的数据与模型代码读明白、**在纸上手算出本周三个 config 的参数量和显存量级**。晚上你要能在不看代码的情况下说清「`prepare.py` 产出的三个文件分别装什么」。手算先于实测是本计划反复用的方法——只有先猜出数字，实测才有意义。

任务 1：清 Week 4 gaps（约 60 分钟）

打开 `week05/day1/gaps.md`（刚从 week04 复制来的），本周**只清下面这几类**，因为它们直接是 M1 的地基；其余留到 Day 7 处理：

1. 「断点续训只跑通过一次，没验证过 resume 后 loss 是否连续」→ 动作：今天写下 Day 2 任务 4 的验证脚本（判据：resume 前后相邻 10 步 loss 波动 <5%）。
2. 「bf16 与 fp16+GradScaler 的区别说不清」→ 动作：用 15 行 numpy/torch 小实验：同一层输入分别转 fp16/bf16，打印 `x.max()` 与溢出位置（bf16 指数位 8 位、fp16 只有 5 位，所以 fp16 容易上溢、必须 scaling）。判据：NOTES 里有一张三行表（指数字位/尾数字位/从零训稳定性）。
3. 「显存解剖表只抄了公式没实测」→ 动作：今天的任务 4 就是它。
4. 「手写 AdamW 与 torch 对拍过了，但没看过 `betas=(0.9,0.999)` 与 `(0.9,0.95)` 的差别」→ 动作：Day 3 排查演练时正好用上（nanoGPT 从零训用的是 `beta2=0.95`，为什么，见 Day 3 注）。

完成判据：gaps.md 里上述 4 条各有 `[x]` 或明确推到本周哪天哪个任务；**至少 3 条今天闭合**。

> 注：别在这一步翻旧代码翻成半天。规则是每个洞最多 15 分钟，超时就把「洞的准确描述」写清楚然后跳过——能被准确描述的洞，一半在写下来的过程中自己就闭合了。

任务 2：读 nanoGPT 的数据与模型代码（约 80 分钟）

1. 打开 `data/shakespeare_char/prepare.py`，通读，回答三个问题（写进 NOTES）：它把原始文本切成什么比例的 train/val？`train.bin` 里存的是什么类型、什么含义的数组？`meta.pkl` 里的 `stoi` / `itos` 干什么用？
2. 打开 `model.py`，只看 `CausalSelfAttention` 和 `Block`，对照你 Week 3 写的 `block.py`，列出 3 处它和你写的不一样。
3. 打开 `train.py`，找到 `get_batch()`：把 `xs = torch.randint(...)` 那两行标出来，说出 `x` 和 `y` 的关系（提示：`y = x[:, 1:] + (0, block_size-1)` 型的错位，这就是「next-token prediction」的全部实现）。
4. 找到 `estimate_loss()` 与 `forward_backward_fn`，确认 eval 用了 `torch.no_grad()`。

完成判据：NOTES 新增 ≥10 条自己的话（不许抄注释）；能回答：为什么 `vocab_size` 等于 `len(itos)` 而不是一个固定超参。

任务 3：本地 CPU 冒烟（约 80 分钟）

在本地（不占卡）用极小 config 跑 200 步，确认整条管线通了：

```
cd E:/study/posttrain/nanoGPT
python data/shakespeare_char/prepare.py
python - <<'PY'
import pickle, numpy as np
m = pickle.load(open('data/shakespeare_char/meta.pkl','rb'))
print('vocab_size =', len(m['itos']), '| itos[:6] =', m['itos'][:6])
a = np.memmap('data/shakespeare_char/train.bin', dtype=np.uint16, mode='r')
print('train tokens =', a.size, '| idx.max =', int(a[:2000000].max()))
PY
```

拿到真实 `vocab_size` 后，**把它取自然对数记下来**（均匀分布时交叉熵的理论值，也是你 Day 2 那条曲线的起点）。然后：

```
python train.py --dataset=shakespeare_char --scale_lr --batch_size=32 --block_size=128 \
  --n_layer=2 --n_head=4 --n_embd=128 --device=cpu --dtype=float32 --compile=False \
  --init_from=scratch --max_iters=200 --eval_interval=100 --log_interval=20 --out_dir=out-local-smoke
```

完成判据：CPU 上 200 步 loss 单调下行且终值比初值低 **≥20%**（预期量级：200 步 4.2 → 3.4~3.6，CPU 约几分钟到十几分钟）；`out-local-smoke/` 下有 `ckpt.pt`。

> 注：Windows 上 `import torch` 报 `OMP: Error #15: ... libiomp5md.dll already initialized`，是 conda 的 MKL 和 torch 自带 OpenMP 打架。正解是用 `.venv` 而不是 conda base；急着往下走就先 `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）或 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）绕过，然后把「切到 venv」记进 gaps。CPU 冒烟看到明显 loss 下降就停，**别在 CPU 上追指标**。

任务 4：参数量与显存手算表（约 100 分钟，本周最有长期价值的一小时）

对下面三个 config，先手算，再上卡实测（Day 2 回填）：

config	n_layer	n_head	n_embd	block_size	batch
A 冒烟	2	4	128	128	32
B 本周主力（字符级）	6	6	384	256	24
C 升级（BPE/TinyStories）	6	6	384	512	12

1. 参数量：decoder-only GPT 的骨干近似 $12 \cdot n_layer \cdot d^2$（注意力 $4d^2$、FFN $8d^2$），加 embedding $vocab_size \cdot d$、LM head 与 wte 权重共享（nanoGPT 是 tied 的，别重复计），加各类 norm/bias（量级可忽略）。
2. bf16 显存解剖：权重 2B/参 + 梯度 2B/参 + AdamW 状态 8B/参（fp32 一阶二阶各 4B）；剩下是激活。
3. 激活粗估：每层约 $batch \times block \times d \times k$，k 取 20~40（QKV/FFN 中间态/softmax 输出/残差，注意力矩阵 $batch \times head \times block \times block$ 单列出来）。
4. **手算 attention 矩阵占比**：config C 里 $12 \times 6 \times 512 \times 512 \times 2B = ?$，和总激活比一下。

完成判据：`week05/day1/memory-estimate.md` 里三行表，每行给出参数量、权重+优化器态字节数、激活估计；**声明你的估算误差范围**（如「±2 倍」）；Day 2 用 `torch.cuda.max_memory_allocated()/reserved()` 回填实测并写差值倍数。

> 注：这表在面试里的用法是——被问「你这个模型占多少显存、瓶颈在哪」时，你能张口报出「参数 11M、优化器态约 180MB、大头是激活」而不是「我不知道，反正跑起来了」。手算的目的在于此。顺带解释 nanoGPT 为什么把 `block_size` 视为一等超参：显存与算力对它是**平方**关系。

任务 5：收工与明日开卡准备（约 40 分钟）

1. 把 Day 2 的命令、tmux 启动方式、关机流程写在 `day2/todo.md`，一行一步。
2. `git add -A && git commit -m "week05 day1: prep + estimates" && git push`。
3. NOTES 三行：今天最卡的一处 / 明天必须先看到什么 / 明晚计划开始时间。

今日验收：gaps 4 条有状态 + CPU 冒烟 loss 下行 + 手算表完成（实测值待回填，允许空白）。三项齐才收工。

完成判据：`day2/todo.md` 存在且第一步是开卡命令；`git log -1` 有今天的 commit；NOTES 三行齐全。

Day 2 • 周二：上卡——字符级莎士比亚正式训练 + 基线对比 + 断点续训

今日总览（6.5h）：今天第一次真正「用租的卡从零训一个模型」。目标很具体：晚上关机前，`wandb` 里有一条完整的 U 型 train/eval 曲线，`out-char6` 里有 checkpoint，采样出的文本一眼能认出是莎士比亚排版，而且你的模型 PPL 比你自己算的 unigram 基线低一大截——「明显优于基线」是 M1 的硬要求。卡时预算 2h。

任务 1：开卡、环境冒烟、数据落地（约 60 分钟）

1. AutoDL → 算力市场 → RTX 4090 24G → 按量计费 → 镜像选 PyTorch 2.x + Python 3.10 + CUDA 12.1。没货就换 3090/4090D/A100，**不要刷页面等卡**（今天的任务不挑卡，只挑显存 ≥16G）。
2. 进 JupyterLab 终端：

```
nvidia-smi
python -c "import torch;print(torch.__version__, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
source /etc/network_turbo
cd /root/autodl-tmp && git clone https://github.com/karpathy/nanoGPT.git && cd nanoGPT
pip install -r requirements.txt -q
python data/shakespeare_char/prepare.py
python train.py --dataset=shakespeare_char --init_from=scratch --device=cuda \
  --n_layer=2 --n_head=4 --n_embd=128 --block_size=128 --batch_size=32 \
  --max_iters=20 --eval_interval=10 --compile=False --out_dir=out-smoke
```

3. 把本地 `week05/day1/` 的笔记 push 到仓库，实例上 `git pull`，之后所有代码改动都在实例上 commit（收工前 push）。

完成判据：第二条命令输出 `True` 且能打印卡名；20 步冒烟无报错；`data/shakespeare_char/` 下 `train.bin` / `val.bin` / `meta.pkl` 三个文件都在。

> 注：`--compile=False` 是本周默认。`torch.compile` 在这台机器上可能因 triton/驱动版本卡几分钟到十几分钟编译，甚至报错；等你主线跑完再打开它测一次速度差，属于加分项，不许为它阻塞当天任务。

任务 2：正式训练 config B（约 150 分钟，含等训练）

1. 用「开始前」的 `run_char.sh`（`--out_dir=out-char6`）启动。**启动前 60 秒盯住第一条 loss**：它应该接近昨天算的 `ln(vocab_size)`，如果一开始就是 `nan` 或比理论值高一倍以上，立刻 Ctrl+C，不要让它烧 10 分钟卡时。
2. 等待期间（预计 30-60 分钟）做任务 3 的基线脚本。
3. 训练每 `eval_interval=500` 会打印一次 eval loss，**手动记 3 个点**（iter、train loss、val loss）到 `day2/curves.md`，并把 wandb 曲线截图存 `m1-nanogpt/figures/`。wandb 连不上就 `export WANDB_MODE=offline`，日志落本地，回家再 `wandb sync` ——这不是失败。
4. 训练到 3000 步左右可以先做一次采样（另开终端）：

```
python sample.py --init_from=gpt2 --out_dir=out-char6 \
  --num_samples=3 --max_new_tokens=400 --temperature=0.8 --seed=42 \
  --start="KING HENRY VIII:" --out-file=/root/autodl-tmp/sample-step3000.txt
```

> 注：`sample.py` 用 `init_from='gpt2'` 时会去读 `out_dir/meta.pkl` 来还原字符表，所以**必须和 checkpoint 同一个 out_dir**；报缺 `meta.pkl` 就把 `data/shakespeare_char/meta.pkl` 拷进 `out-char6/`（nanoGPT 的设计如此）。

完成判据：`out-char6` 最终 `val loss < 1.9`（预期量级：6 层 384 维、6000 步在 24 条 batch 下约 1.5-1.75；`<1.9` 是及格线，别追更低）；采样文本至少 3 段**满足**：绝大多数行首是「角色名+冒号」的排版、句子内部是连续字母加换行、读起来像台词而非随机字母。三项里缺任何一项 → 进 Day 3 的排查演练（这正是本周设计好的衔接）。

任务 3：基线对比脚本——证明你的模型不是运气（约 80 分钟）

写 `day2/baselines.py`（本地或卡上跑都行，它是纯 numpy）：

```
import math, pickle, numpy as np
rng = np.random.default_rng(0)                        # 种子写死，别人重跑同一个数
V = len(pickle.load(open('data/shakespeare_char/meta.pkl','rb'))['itos'])
tr = np.memmap('data/shakespeare_char/train.bin', dtype=np.uint16, mode='r')
va = np.memmap('data/shakespeare_char/val.bin',   dtype=np.uint16, mode='r')
N = min(400_000, va.size - 1)
val = np.asarray(va[:N+1])

cnt = np.bincount(np.asarray(tr[:1_000_000]), minlength=V).astype(np.float64)
p = (cnt + 1.0) / (cnt.sum() + V)                     # 拉普拉斯平滑，见过一次的 token 不至于 0
def ce(probs_fn):
    tot = 0.0
    for i in range(0, N, 20_000):
        chunk = val[i:i+20_000]
        lp = np.log(probs_fn(chunk))
        tot += -lp[np.arange(chunk.size), chunk[1:]].sum()
    return tot / N
random_loss = ce(lambda c: np.full(V, 1.0 / V))     # 常数：应等于 ln(V)
unigram_loss = ce(lambda c: p)
print(f'random : loss {random_loss:.4f}  ppl {math.exp(random_loss):.2f}')
print(f'unigram: loss {unigram_loss:.4f}  ppl {math.exp(unigram_loss):.2f}')
```

再补一段「bigram 基线」（用 train 的相邻对计数 + 平滑，查表算 val 的 CE）。三个数 + 你的模型，写成一张四行表放进 README。

完成判据：`random` 那行打印的 loss 与 `math.log(V)` 差 <0.01（**这是自检，说明你 CE 写对了**）；表格里模型 PPL 至少比 unigram PPL 低 **2 倍**（预期量级：unigram PPL ≈15-20、模型 PPL ≈4-6）。若低不到 2 倍，把它记成 Day 3 的排查对象，不要粉饰。

> 注：为什么非要自己算基线？因为「loss 1.55 好不好」这句话没有参照物就是空话。面试官追问「你怎么说明模型学到了结构而不只是字符频率」，你甩出 bigram 基线和模型 PPL 的差，就是回答。另外：bigram 基线的实现要点是全量频率表 + 平滑，**别在 20000 条样本上算条件概率表**，那会让 bigram 虚高得像噪声，你会误判。

任务 4：断点续训演练（约 60 分钟）

1. 训练跑到中途（如 2000 步）时 `Ctrl+C` 杀掉，或让 Day 4 那次长跑自然中断——目标是练 resume。
2. 用同一条命令只改一处：`--init_from=resume --out_dir=out-char6`，重启。
3. 检查续上：iter 编号从 2000 继续、重启后第一条 loss 与中断前最后一条波动 <5%、优化器状态存在（`ckpt['model_ckpt']` 与 `ckpt['optimizer']` 非空）。

```
python - <<'PY'
import torch
ck = torch.load('out-char6/ckpt.pt', map_location='cpu')   # weights_only=False 才能加载 optimizer 对象
print('iter =', ck['iter_num'], '| keys =', sorted(ck.keys()))
print('optimizer states =', len(ck['optimizer']['param_groups']) > 0)
PY
```

完成判据：上面脚本打印出非零 `iter_num`；resume 后曲线无跳变（<5%）；`day2/resume.md` 写清 resume 改了哪三个字段（`iter_num` / `best_val_loss` /优化器态）。

任务 5：归档 + 关机核对（约 40 分钟）

采样文本、曲线截图、`ckpt` 路径、wandb 链接，全部写进 `m1-nanogpt/day2-results.md`；`git push`；把 checkpoint 从持久盘留一份（24G 卡的空间足够）。**收工前在控制台点关机并确认显示「已关机」**，把实际卡时/金额回填 `cost.md`。

今日验收：val loss <1.9 + 采样文本 3 段入库 + 基线四行表 + resume 无跳变 + 卡时 ≤2h 且已关机。

完成判据：`day2-results.md` 含采样文本路径、曲线截图、ckpt 路径、wandb 链接四项；`cost.md` 今日行填满；AutoDL 控制台显示「已关机」（记一句时间即可）。

Day 3 • 周三：训练不稳定排查演练（M1 自测标准，本周不许跳）

今日总览（6.5h）：今天做一件反直觉的事——**故意把训练搞崩，再按固定顺序把它定位出来**。顺序是写死的：**学习率 → 数值精度 → 数据 → 初始化**。晚上你要交出一份 `diagnosis.md`，里面每个结论都对应一个你自己跑出来的数字。面试问「loss 变 NaN 你怎么办」，从今天起你答的是自己做的事。卡时预算 3h，三次 run 全是短跑（各 300-800 步）。

任务 1：制造三种「不健康」并记录现象（约 100 分钟，卡时约 1h）

统一骨架：小 config（`n_layer=4 n_embd=256 block_size=256 batch=24`）、`max_iters=800`、`log_interval=20`、`seed` 固定，每次只改一个变量，跑三组：

1. run-LR：`--learning_rate=0.03`（正常是 6e-4~1e-3）
2. run-PREC：`--dtype=bfloat16`，`--learning_rate=1e-2`（让精度问题更容易暴露）；对照组 `--dtype=float32` 同 lr
3. run-DATA：把 `train.bin` 里随机 2% 的 token 改成 `vocab_size`（越界值）或改成随机值，看崩法；数据脚本：

```
import numpy as np, shutil
shutil.copy('data/shakespeare_char/train.bin','data/shakespeare_char/train_clean.bin')
a = np.memmap('data/shakespeare_char/train.bin', dtype=np.uint16, mode='r+')
rng = np.random.default_rng(7)                    # 种子写死，破坏可复现
idx = rng.integers(0, a.size, size=int(a.size*0.02))
a[idx] = rng.integers(0, 200, size=idx.size)      # 制造异常 token
a.flush(); print('poisoned tokens:', idx.size)
```

每组都要打印三样：**loss、grad_norm、`torch.cuda.max_memory_allocated()`**。grad_norm 的取法（改 `train.py` 一行，别忘记录进差异清单）：

```
grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), config.grad_clip)
if it % 20 == 0:
    print(f'grad_norm {grad_norm:.4f}')
```

完成判据：三组各留下一条明确现象描述 + 出现步数：LR 组的 loss 上行或 `nan` 的首个 iter；PREC 组在什么 lr 下 bf16 与 fp32 表现开始分岔；DATA 组 loss 是否 `nan`、以及**是否越界触发 CUDA device-side assert 或采样出现乱码**（两种都算观察到，写清实际看到哪个）。

> 注：越界 token 触发的是 CUDA 侧断言，报错信息往往指向 `Embedding` 或 `cross_entropy`，而**真正的根因在数据管线**。这是新手最容易误判的一类：报错位置≠故障位置。你要在 NOTES 里给自己写下这句话。

任务 2：按 LR→精度→数据→初始化 定位演练（约 140 分钟）

规则：每一步只改一个变量，其余全部固定，并把「固定了什么」写进诊断表。 对任务 1 的每个 run 走一遍完整流程：

1. LR：把学习率降 10 倍，同 seed 重跑 800 步 → 恢复则记「LR 类」；不恢复进下一步。
2. 数值精度：`bfloat16 → float32` 重跑 → 恢复则记「精度类」。注意区分两种失败模式：bf16 从零训**发散**（梯度累加精度不够）与 fp16 **上溢**（`inf/nan`，需要 GradScaler/loss scaling）。写清你遇到的是哪一种。
3. 数据：`train_clean.bin` 换回去重跑 → 恢复则记「数据类」；顺手加两行断言 `assert idx.max() < vocab_size` 进 `get_batch()`。
4. 初始化：nanoGPT 的 scaled init（残差投影按 `0.002/√(2·n_layer)` 缩放、`wte/wpe` 用 `0.02` 正态、`Linear` 权重 `uniform(-k,k)` 其中 `k=sqrt(1/n_embd)`）全部换成 PyTorch 默认，同 seed 重跑 → 对比前 300 步。 先备份 `cp model.py model.py.bak`；诊断结束用 `sed -i 's/ def _init_weights(self, module):/ def _init_weights_unused(self, module):/' model.py` 之类的方式让默认初始化生效（写法不唯一，能确认真的生效即可，判据里有检查动作）。

完成判据：`diagnosis.md` 里三张定位表，每张 ≥4 行（四步各一行：改了什么 / 看到什么 / 结论），**三个 run 各自收敛到一个唯一根因**，其中 LR 那个必须有明确的「恢复用的 lr 值」。

> 注：这个顺序不是玄学，是成本顺序：lr 改一个字符（30 秒）、dtype 改一个字符（30 秒）、数据要重跑 prepare（几分钟）、初始化要动模型代码（最贵）。排查顺序 = 改动成本从低到高。面试讲这句就够，别背清单。
> 注：顺手回答一个 Day 1 留的问题——nanoGPT 从零训为什么把 `beta2` 从 0.999 降到 0.95：AdamW 的二阶动量窗口 `1/(1-β2)` 分别是 1000 步和 20 步；小模型从零训练的梯度尺度变化很快，长窗口会让分母估计滞后。写进 NOTES，这是面试官爱挖 second-moment 语义的地方。

任务 3：给训练加两个监控（约 60 分钟）

1. 在 `train.py` 里加：`grad_norm` 曲线（任务 1 已加）+ **loss spike 检测**（连续 20 步内 loss 比滚动均值高 >2 倍就打印 `[SPIKE]` 并 `wandb.log({'spike': 1})`）。
2. 把 Day 2 的正式 run（`out-char6`）在加监控后重跑 500 步确认无回归（预期：无 spike 报告）。
3. 打印 `torch.cuda.max_memory_allocated()` 与 `reserved()`，回填 Day 1 的手算表（差值倍数、以及「哪一项被低估了」的一句话结论）。

完成判据：`[SPIKE]` 检测在 run-LR（大 lr）上**至少触发一次**、在正常 run 上**零误报**（各跑一次，把两个数字写进日志）；手算表实测列填满，结论一句话。

任务 4：写「面试可讲版」诊断话术（约 60 分钟）

把 `diagnosis.md` 压成 200 字 + 4 个要点，格式：现象 → 我先排什么 → 判据是什么 → 结论 → 怎么防。放进 `NOTES.md` 和一个新文件 `m1-nanogpt/TROUBLESHOOTING.md`。同时写清三件真实数字：本次花了几小时卡时、多少钱、最好的一次 run 的 val loss。

完成判据：对镜 2 分钟讲完这段，录音回听一遍；`TROUBLESHOOTING.md` ≥300 字且含 4 个步骤名（不许写「逐步排查」这种含糊词）。

任务 5：收工与记账（约 30 分钟）

回填 `cost.md`（今天计划 3h，实际若超 4h 就把 Day 4 长跑的 `max_iters` 从 6000 砍到 3000）。push + 关机确认。

今日验收：三张定位表齐全 + spike 检测有效 + TROUBLESHOOTING.md 成文 + 手算表实测回填。

完成判据：`cost.md` 今日行填满（计划 3h / 实际 ?h）；三次发散 run 的启动命令存进 `m1-nanogpt/scripts/`（别人能重跑）；已关机。

Day 4 • 周四：升级到 BPE 与 TinyStories，挂上本周的长跑

今日总览（6h）：字符级模型再漂亮也只是玩具，因为它的「词」是字母。今天你要把 token 换成真正的 BPE，并把语料换成 TinyStories——一个专为「从零训出能连贯讲故事的模型」设计的数据集。晚上睡前，你的第一次 BPE 大模型跑要有一个可信的中间状态（loss 明显低于 `ln(V)`）并把 resume 命令备好。**记住铁律：睡前一定要关机，不要为了省事让卡跑一整夜**（8 小时 ≈¥16-20，直接把本周预算吃穿）。卡时预算 2.5h。

任务 1：读懂 openwebtext 管线并写自己的 tinystories prepare（约 120 分钟）

1. 通读 `data/openwebtext/prepare.py`，回答：它怎么下载数据？分块编码（`process_text_chunk`）为什么必需而不是把所有文本读进内存？产出的 `train.bin` / `val.bin` 是什么 dtype？为什么这条路径**不需要** `meta.pkl`（因为 BPE 编解码由 tokenizer 提供，`vocab_size` 来自编码器的 `n_vocab`）？
2. 下载 TinyStories（HuggingFace 数据集 `roneneldan/TinyStories`，走镜像）：

```
pip install datasets pyarrow -q
export HF_ENDPOINT=https://hf-mirror.com
python - <<'PY'
from datasets import load_dataset
ds = load_dataset('roneneldan/TinyStories', split='train')
with open('tinystories_raw.txt','w',encoding='utf-8') as f:
    for i, r in enumerate(ds):
        if i >= 40000: break                        # 前 4 万篇做本周规模
        f.write(r['text'].strip() + '\n\n')
print('wrote', 40000, 'stories')
PY
```

下载失败/镜像不通的降级顺序（不许卡在第一步）：① `source /etc/network_turbo` 再试一次；② ModelScope 搜 TinyStories；③ **本周降级为只交字符级模型**（M1 允许），把 BPE 部分作为「冒烟 + 曲线」入博客，并把数据下载推到 Day 5 早上。

3. 写 `data/tinystories/prepare.py`：**照 openwebtext 的结构抄，不要发明新写法**，只改三处：数据源换成 `tinystories_raw.txt`、切分比例 `99.0/1.0`、验证集从**整篇故事**边界切（别从一篇故事中间切，否则 val 里全是见过的前半句，PPL 会假低）。
4. 在 `train.py` 里注册这个数据集：搜 `dataset == 'openwebtext'` 找到数据分支，把它改成同时接受 `tinystories`（`elif dataset in ('openwebtext', 'tinystories'):`）。

完成判据：round-trip 测试通过——脚本打印 `decode(encode(样本)) == 样本` 完全一致；`idx.max()` 严格小于编码器词表大小；train token 数写入 NOTES（**量级预期：4 万篇故事约 5×10^7 token**，实际以你的脚本输出为准）。

> 注：这一步最烧时间的不是代码，是「你抄的是哪版仓库」。nanoGPT 的 openwebtext prepare 在不同版本里字段名不一样。**先跑通仓库原版一次（哪怕只处理 1000 篇），再改成你的**，比对着 README 猜要快。

任务 2：BPE 冒烟 run（约 70 分钟，卡时约 0.5h）

先确认初始 loss 的理论值：`import math; print(math.log(50257))` ≈ 10.8（你的编码器词表若是 50304 就按它算）。

```
python train.py --dataset=tinystories --init_from=scratch --scale_lr \
  --batch_size=24 --block_size=256 --n_layer=4 --n_head=6 --n_embd=384 \
  --learning_rate=3e-4 --max_iters=300 --eval_interval=100 --log_interval=25 \
  --decay=True --warmup_iters=20 --beta2=0.95 --grad_clip=1.0 \
  --dtype=bfloat16 --compile=False --out_dir=out-ts-smoke
```

完成判据：300 步内 loss 相对首条 eval 降 **≥30%**；无 `nan`、无 spike；把首末两个数记进 `day4/smoke.md`。没到 30% 就当场走 Day 3 的四步排查（lr → dtype → 数据 → 初始化），把过程写进 diagnosis.md 追加节。

任务 3：正式长跑首段 + 可续训安排（约 80 分钟，卡时约 1.5h）

1. 写好 `run_ts.sh`：config C（`n_layer=6 n_head=6 n_embd=384 block_size=512 batch_size=12`、`learning_rate=3e-4`、`max_iters=6000`、`warmup_iters=400`、`lr_decay_ratio=0.1`、`decay=True`、`beta2=0.95`、`grad_clip=1.0`、`dtype=bfloat16`、`eval_interval=500`、`out_dir=out-ts6`）。
2. 用 tmux 启动，**盯前 200 步**：loss 从 ≈10.8 掉到 5-7 属正常量级；若卡在 10.8 附近不动，立刻停（多半是 `data/tinystories` 文件错位或 batch 的 y 没错位）。
3. 睡前（约跑满 1.5h）主动 `Ctrl+C`，关机，把 Day 5 的 resume 命令写进 `day5/todo.md`。

完成判据：首段 ≥1000 iter 且 eval loss ≤3.0（预期量级 2.2-2.8）；`out-ts6/ckpt.pt` 落在持久盘；关机后控制台显示「已关机」；今天卡时 ≤2.5h 且已回填 `cost.md`。

> 注：主动中断再 resume 看着像浪费，实际是**这周你唯一一次在真实预算约束下练 checkpoint/续训**——线上训练天天如此，没人敢跑一个不能安全中断的任务。Day 2 你练过 resume，今天用第二次，就成肌肉记忆了。

任务 4：写「与 Karpathy 实现的差异清单」（约 60 分钟）

打开 `git diff`（或 `diff model.py model.py.bak`、`diff train.py 原始版`），逐条列出本周你对仓库动过的每一处：数据分支、`tinystories/prepare.py`、grad_norm 打印、spike 检测、断言、初始化实验。**每条写「改了什么 / 为什么 / 是否影响主线结果」**。这份清单直接进博客和 README，是面试诚信声明的原材料（哪些是复现、哪些是我写的）。

完成判据：清单 ≥5 条，且每条都能用一行命令验证（`git diff --stat` 或文件路径）。

任务 5：收工笔记（约 30 分钟）

今日验收：round-trip OK + 冒烟达标 + 长跑首段 eval ≤3.0 + 差异清单 ≥5 条 + 已关机。

完成判据：`day5/todo.md` 里的 resume 命令完整到可直接粘贴（含 `--init_from=resume --out_dir=out-ts6`）；今天卡时 ≤2.5h、本周累计 ≤7.5h，超出就把 `max_iters` 砍到 3000 并写明。

Day 5 • 周五：续训收口 + 采样对照 + 把 repo 做成能交给别人的样子

今日总览（6.5h）：把 TinyStories 的长跑跑完，然后做两件「产品化」的事：一张能说明采样参数怎么影响输出的对照表，和一个别人 `git clone` 就能跑起来的 `m1-nanogpt`。今天卡时要压住（2.5h）：**开卡 → resume → 去写 README → 回来采样 → 关机**。

任务 1：resume 收尾 + 正式 eval（约 60 分钟，卡时约 1.5h）

1. 开卡，`--init_from=resume --out_dir=out-ts6` 续到 6000 iter（或按计划砍到 3000 收工）。
2. 训练完成后跑一次干净 eval：用 `train.py` 打印的 `final eval loss`，自己算 PPL = `exp(loss)`；**再对字符级模型 `out-char6` 做同样的事**，两个模型的 val loss / PPL 并排写进表。

```
python - <<'PY'
import numpy as np, math
B = np.memmap('data/tinystories/val.bin', dtype=np.uint16, mode='r')
n = min(1_000_000, B.size - 1)
a = np.asarray(B[:n+1].astype(np.float64))
print('val tokens =', n)                    # 记下来，别人要能核对你的 PPL 分母
PY
```

完成判据：两个模型的 val loss + PPL 数字入库（**PPL 必须给出分母 token 数**）。**不许把两个模型的 PPL 并排当结论**——词表不同，不可比。

> 注：**不同 tokenizer 的 loss/PPL 不能横向比**，这是最常被面试官抓的一条。字符级 vocab 65、BPE vocab 5 万，`ln(V)` 起点都差 6 倍。本周的正确写法是：字符级模型比 bigram/unigram 基线，BPE 模型比它自己的 unigram 基线，两者只比「相对基线的降幅」。

任务 2：采样参数对照表（约 150 分钟，卡时约 1h）

对 `out-ts6` 跑一个 3×2 网格（temperature 0.6/0.9/1.3 × top_k 0/200），每格 2 条样本、固定 seed，起始 prompt 统一：

```
for T in 0.6 0.9 1.3; do for K in 0 200; do
  python sample.py --init_from=gpt2 --out_dir=out-ts6 \
    --num_samples=2 --max_new_tokens=250 --temperature=$T --top_k=$K --seed=42 \
    --start="Once upon a time there was a little girl" \
    --out-file=samples/t_${T}_k${K}.txt
done; done
```

统计每格两条样本的：平均句长、换行/句号密度、重复 3-gram 占比（复用 `day2` 写的工具函数）。**字符级模型 `out-char6` 用同一个提示跑一个 3 格子集**（预算紧就 2 格）。

完成判据：表格 ≥6 格且每格有 2 条真实文本入库；用一句话说明「t→0」与「t→1.3」各自的退化表现，**并能在样本里指出至少一处证据**（如 t=0 时的循环句、高 t 时的破句）。

> 注：top_k 与 temperature 谁在前？nanoGPT 的实现是先除温度再 softmax 再截 top-k 再重归一化，所以 top-k 生效于已展平/已锐化的分布上。追问「t=0 会怎样」——`logits/0` 除零，实测行为是变成 `inf` 或抛错，**别写「等价于贪心」**；要看代码或实测再说。

任务 3：挑 checkpoint + 失败案例 3 条（约 60 分钟）

从采样表里挑 3 条最差样本（重复、跑题、无标点），每条标注：模型、step、temperature、失效模式一句话。这 3 条是博客和 Day 7 验收表的必需项（「有失败案例」比「全是好消息」可信得多）。

完成判据：`m1-nanogpt/failures.md` 3 条齐全，每条有可复现命令（含 seed）。

任务 4：整理 `m1-nanogpt` 仓库（约 90 分钟）

目录定稿（缺一项视为今天没完成）：

```
m1-nanogpt/
  README.md              # 见下面四段要求
  configs/char6.md  tinystories6.md      # 完整命令行 + 参数量手算与实测
  scripts/run_char.sh  run_ts.sh  baselines.py  sample_grid.sh
  figures/  loss_char.png  loss_ts.png  memory.png
  samples/  char/*.txt  ts/t*_k*.txt
  TROUBLESHOOTING.md  failures.md  COSTS.md
  DIFFS-VS-NANOGPT.md    # Day 4 的差异清单
```

README 必须四段：① 一句话这是什么 ② 三条命令复现（准备数据 → 训练 → 采样）③ 结果表（val loss / PPL / 基线对比 / 卡时与花费）④ 声明：模型结构与训练脚本基于 Karpathy 的 nanoGPT，我自己写了 A/B/C，我改动的部分见 `DIFFS-VS-NANOGPT.md`。

完成判据：`COSTS.md` 里本周累计卡时 ≤12h、金额 ≤¥30（超了就写明超因，别改数字）；README 四段齐全且含 `ckpt` 的 sha256 或大小（别人核对用）；`git log --oneline` 里每天有 ≥1 个 commit。

任务 5：关机 + 收工笔记（约 30 分钟）

今日验收：两模型 PPL 表 + 采样对照表 ≥6 格 + 失败案例 3 条 + repo 目录齐全 + 已关机。

完成判据：本周累计卡时 ≤12h 且 `COSTS.md` 已填；控制台显示「已关机」；NOTES 新增 ≥4 条（采样表里实际看到的现象必在其中一条）。

Day 6 • 周六：闭卷自测 + 博客《我从零训了一个 GPT》发布

今日总览（6h）：Phase 1 的知识验收 + 公开输出。上午闭卷做 4 道题（不看任何资料，包括你自己的 NOTES），下午把三周的素材写成博客并发布。**先做题再写博客**——题目做不出来的地方，正是博客该写清楚的地方。

任务 1：闭卷自测 4 题（约 60 分钟，计时）

写在一个新文件里，不查资料，到点停笔：

1. 用 NumPy 写 scaled dot-product attention 的完整前向，输入形状 `(B, T, C)`，多头 `h`、每头 `d = C/h`，包含：切头用的 reshape/transpose、缩放、causal mask、softmax、合并头。**标出每一步的张量形状**。
2. 白板默写一个 decoder-only Block（Pre-LN 版）的数据流，写出 FFN 内部维度变化，并说明为什么残差分支的投影要做小尺度初始化。
3. 语言建模为什么用交叉熵？写出它与 KL 的关系；`PPL = exp(CE)` 的推导一句话说明「每步有效分支数」的含义。
4. 训练 loss 变成 NaN，你的四步定位顺序是什么？每步看哪个指标？（这题不许背，昨天你亲手跑过三次）

完成判据：4 题全部写完（**空着才算失败**）；第 1 题的代码事后拷进 `day6/attn_review.py` 并用 `(2,5,16)` 随机输入跑通、上三角权重为 0 的断言通过。

任务 2：博客成文（约 180 分钟）

专栏《从全栈工程师到后训练》第 2 篇（第 0 篇是计划篇，第 1 篇是 micrograd），标题《我从零训了一个 GPT》。2000-3000 字，结构写死，不许自由发挥：

1. 开头 200 字：为什么非要从零训一个 GPT（不训它，后面所有 SFT/RL 都是玄学）。
2. 数据与 tokenizer：字符级 vs BPE，为什么 PPL 不能横向比（把 Day 5 那条注写成一段，这是本文最硬核的段落）。
3. 训练：config、参数量手算 vs 实测表、loss 曲线截图、卡时与花费（真实数字）。
4. **我把训练搞崩了三次**：三次现象 + 四步定位（贴 `grad_norm` 曲线一张），这一段是全文记忆点。
5. 采样参数对照：3 张截图或一段文本对比，说明 temperature/top-k 各自买到什么。
6. 失败案例 3 条（诚实贴坏样本）。
7. 与 Karpathy 实现的差异声明（列 `DIFFS-VS-NANOGPT.md` 要点）。

8. 下一步：SFT。一句话预告。

完成判据：正文 ≥2000 字、图 ≥3 张、含真实数字（loss/PPL/卡时/金额），**全文没有一句无法验证的形容词式结论**（如「效果很好」——必须写成「val loss 1.62，比 bigram 基线低 X 倍」）。

任务 3：发布与归档（约 40 分钟）

知乎发布，链接贴 `NOTES.md` 顶部和 `m1-nanogpt/README.md` 第一段。顺手把 `figures/` 与采样片段上传到仓库（博客图用仓库 raw 链接，避免二次丢失）。

完成判据：链接可匿名访问（手机浏览器打不开登录态也能看）。

任务 4：gaps 更新与向周 6 映射（约 50 分钟）

新建 `week06/gaps-from-week5.md`，把本周新暴露的洞按「错了什么 → 反映哪个空洞 → 补在哪天 → 用什么动作」写全。**本周必须落到周 6 的三类洞**：

洞	归到	动作
只训过从零续写的 LM，没接触过「对话格式」的数据	周 6 Day 1/Day 2	用 `apply_chat_template` 打印真实字符串，并手写一版对比
没验证过 loss masking（哪些 token 参与 loss）	周 6 Day 2	3 轮对话样本，打印 `label=-100` 的 span 并断言
不知道 lr/warmup 在微调语境下该取多大（本周是 LM 从零训的值）	周 6 Day 3/Day 4	LoRA 用 1e-4 量级、从零训是 3e-4~6e-4，各跑一次并记录
采样/PPL 评测只在自己模型上做过，没有固定评测集	周 6 Day 3	冻结自建 20 题 `eval20.jsonl`（不许抄公开评测集）

完成判据：表里 ≥4 行且每行有动作；**Phase 1 遗留未闭合的洞全部写进来**，不许留在口头。

任务 5：收工笔记（约 30 分钟）

今日验收：闭卷 4 题写完 + 第 1 题代码跑通 + 博客已发布（有链接）+ gaps 映射表 ≥4 行。

完成判据：`gaps-from-week5.md` 已 push；NOTES 顶部有博客链接；记一条「发布后 24 小时内要回复的追问」（没有就写「无」）。

Day 7 • 周日：Phase 1 总验收 + 下周预习

今日总览（4.5h）：今天不排新训练，**不开卡**。上午补漏 + 闭卷复测，中午做 Phase 1 的总验收（这一张表是 Phase 1 的结业证），下午 40 分钟预习 Phase 2。

任务 1：补漏（约 90 分钟）

从 `week06/gaps-from-week5.md` 和本周 `gaps.md` 里各挑 1 个洞，**合计最多 2 个**（贪多必失）。优先补这两类：Day 6 闭卷第 1/2 题错的地方、Day 5 那条「PPL 不能横向比」还没讲顺的地方。

完成判据：补的洞 ≤2 个，每个留一条证据（重跑命令 + 输出数字，或重做的默写），gaps 条目状态从 `[ ]` 变 `[x]`。

任务 2：闭卷复测（约 60 分钟）

重新出一遍 Day 6 的第 1、2 题（换了文件、不许看昨天答案），限时 30 分钟；另外口述第 4 题（四步定位）录 2 分钟音。

完成判据：attention 题形状全对、mask 加在 softmax **之前**、缩放用的是每头维度 `d` 的平方根而不是 `C`；Block 默写里 Pre-LN 位置、FFN 的 `C→4C→C`、残差三条全对。**两题各 ≥80% 正确 = 通过**，未通过就把对应题写进周 6 Day 7 的复测清单（但 M1 的交付物不许因此延期）。

> 注：复测是整个 Phase 1 唯一一次「隔了三天再写一遍」的检验。只看着昨天的卷子点头不算数——会做和三天后还会做是两件事，跟你的训练经验无关，跟遗忘曲线有关。

任务 3：Phase 1 总验收（约 40 分钟）

逐项勾选，结果写进 `week05/review.md`：

#	验收项	判据
1	M1 训练	`out-char6` val loss <1.9；`out-ts6`（或降级后的冒烟 run）eval loss ≤3.0 且 PPL 有分母 token 数
2	基线对比	自建 random/unigram/bigram 三基线，模型 PPL 相对降幅有数字；不同 tokenizer 未横向比
3	不稳定排查	`diagnosis.md` 三张四步定位表；`[SPIKE]` 检出一误报零；`TROUBLESHOOTING.md` ≥300 字
4	训练系统	手算 vs 实测显存表已回填（差值倍数 + 被低估项）；resume 曲线波动 <5%
5	采样与评测	采样对照表 ≥6 格 + 高/低 t 各一处失效证据；失败案例 3 条
6	公开输出	博客链接可访问，含真实 loss 曲线与 3 个踩坑；差异声明在 README
7	闭卷能力	attention + Block 复测通过；四步定位能口述 2 分钟

≥6 项通过 = Phase 1 合格（硬里程碑达成）；不足 6 项，缺的必须在周 6 Day 1 前 2 小时内闭合，且 `review.md` 写清「哪一项、为什么、怎么补」。

完成判据：表格 7 行全部有勾选或缺项补齐日期；每行的证据能在 `m1-nanogpt/` 里指到具体文件。

任务 4：周 6 预习（约 40 分钟）

1. 读 InstructGPT 的后训练三阶段那一节（不看数学细节，只记「SFT → RM → PPO」各自解决什么，写 5 行）。

2. 本地装 Phase 2 依赖并确认版本能共存（这一步做扎实，周 6 第一晚就不烧在环境上）：

```
source E:/study/posttrain/.venv/Scripts/activate
pip install -U transformers datasets accelerate peft trl bitsandbytes safetensors
python - <<'PY'
import transformers, peft, trl, torch
print('transformers', transformers.__version__, '| peft', peft.__version__, '| trl', trl.__version__)
print('cuda', torch.version.cuda, '| bf16 ok', torch.cuda.is_bf16_supported() if torch.cuda.is_available()
else 'no gpu')
PY
```

完成判据：三个版本号打印出来且无 import 报错；`week06/` 目录与 `gaps-from-week5.md` 已存在。

> 注：trl 与 transformers 的版本组合是 Phase 2 的头号环境坑（`SFTTrainer` 的参数名在大版本间改过，例如 `max_seq_length` 移到 `SFTConfig` 里）。现在把版本记下来，遇到报错先查是不是签名搬家，而不是怀疑自己写错了 loss。

任务 5：Phase 1 结课小结（约 40 分钟）

`review.md` 写五块：① Phase 1（周 2-5）四张硬产出清单 ② 真实时间花销 vs 计划（按周记四个数）③ 三周里最有价值的一件事 ④ 仍虚的地方（诚实列 3 条）⑤ 一句话回答「为什么后训练岗要求会预训练侧的东西」。

今日验收：验收表 ≥6 项打勾 + 复测通过 + 周 6 环境 OK + review.md 五块齐全。

完成判据：五块齐全，第 ② 块的四个周时间数字与每周 `review.md` / `COSTS.md` 一致（不许估整）。

三、常见问题与纪律

- **M1 延期了怎么办？** 不允许「延期」，只允许「降级」。降级顺序：先把 TinyStories 砍成冒烟曲线（交付物写成「字符级 GPT 完整 + BPE 管线打通」，这仍然满足 M1 的定义），再砍采样对照表（6 格 → 3 格）。**博客与 Day 7 验收表永远不砍**，它们是里程碑本身。
- **抢不到 4090？** 换 3090（24G）或 4090D，config 只降 batch 不降 block_size（注意力是平方成本）。实在只有 16G 卡：字符级 config 完全跑得动，BPE 版把 `batch_size` 降到 6、开 gradient checkpointing。**任何时候不要为了抢卡换 A100**——本周的模型不吃显存，只吃时间。
- **loss 到不了 1.9？** 先做三件事：确认 `scale_lr=True` 生效（等效 batch 变了 lr 就得跟着变）、确认数据是 90/10 切分且 `val.bin` 不是 `train.bin` 的前缀（截断错位会让 eval 泄漏）、确认 `block_size` 与训练时一致。三件都排掉再走 Day 3 的四步。到不了就报「实测 X，未达判据」并写进 gaps，不许改判据。
- **卡时超了？** 立刻把 `cost.md` 的实际值填上并在 NOTES 写一行原因（多半是忘关机或反复重跑同一个 run）。重跑规则：**任何探索性 run 用 `max_iters ≤ 500` 的短跑**，本周三次发散 run 都是 800 步以内。
- **能不能让 AI 帮我写博客？** 可以帮你顺句子、检查术语，但 loss 曲线、采样片段、失败案例必须是你自己跑出来的真数字。面试时会被追问「这个 spike 在第几步、你当时改了什么」，编不出来。
- **数据与许可**：nanoGPT 仓库是 MIT，莎士比亚文本本身是 public domain；TinyStories 数据集**发布时的许可要按数据集页面标注写进 README**，本周只做内部训练用途，不许把原始语料打包进你自己的仓库分发。这条习惯从本周开始养成，周 7 是硬要求。

- **别提前跳到 SFT。** 本周结束后你会很想立刻去跑 `SFTTrainer`——忍住，周 6 Day 1 有它的位置。提前跳的后果是 chat template 和 loss masking 两个坑你在没准备好验证手段时踩，然后一周后重学一遍。